

Data Preprocessing: Transforming Raw Data into Machine Learning-Ready Data

Presented by
Aditya Krishnan S
I92424069

Supervised by
Dr. M. Krishnaraj
Department of CSE



Why Data Preprocessing?

What is Raw Data?

Data collected directly from the real world - surveys, sensors, transaction logs, web forms - before any cleaning or structuring. It is often incomplete, inconsistent, and unorganized.

Why It Matters

- Machine learning models learn patterns - noisy or missing data teaches wrong patterns
- “Garbage in, garbage out”: model quality is bounded by data quality
- Consistent, clean, numeric-ready data enables every downstream algorithm

Preprocessing typically consumes 60–80% of a data scientist’s project time - more than model building itself.

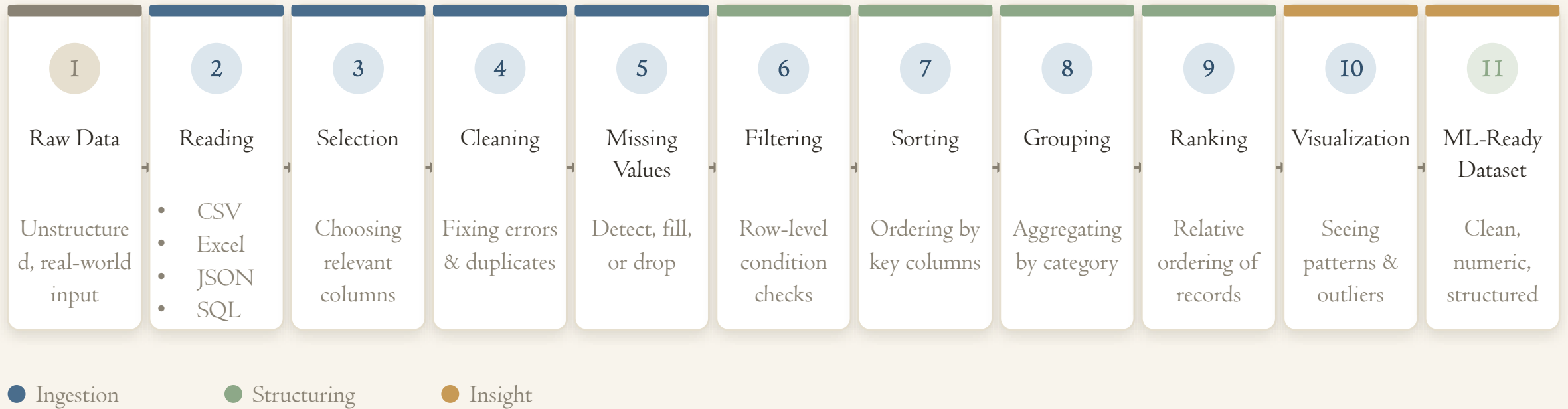
A REAL-WORLD ANALOGY

Cooking a Meal

- 1 Raw ingredients
Raw dataset - unwashed, unsorted, uneven
- 2 Washing & peeling
Cleaning - removing dirt, duplicates, errors
- 3 Chopping to size
Transformation - scaling, encoding, formatting
- 4 Plating the dish
ML-ready dataset - organized and consumable

Just as a great dish depends on ingredient prep, a great model depends on data prep.

The Complete Data Preprocessing Pipeline



This eleven-stage flow is the backbone of the entire presentation - every following section deep-dives into one or more of these stages, using pandas and matplotlib.

Reading Data into Pandas

C

CSV`pd.read_csv()`

Plain-text, comma-separated the most common exchange format

E

Excel`pd.read_excel()`

Spreadsheets with multiple sheets, formulas & formatting

J

JSON`pd.read_json()`

Nested, semi-structured data from APIs & web services

S

SQL`pd.read_sql()`

Query results pulled directly from a relational database

```
● ● ● read_data.py
import pandas as pd

# Read from a CSV file
df = pd.read_csv('students.csv')

# Preview the first 5 rows
df.head()

# Structure, types & memory usage
df.info()
```

READING THE OUTPUT

`df.head()`

Shows the first 5 rows - a quick sanity check of columns and values

`df.info()`

Lists column names, data types, and non-null counts in one summary

Always inspect a dataset immediately after reading it - before writing a single transformation.

Handling Missing Data

Four Core Techniques

- 1

Detect
Count missing values per column
`df.isnull().sum()`
- 2

Remove
Drop rows/columns with nulls
`df.dropna()`
- 3

Fill
Replace nulls with a constant or method
`df.fillna(value)`
- 4

Impute
Statistically-informed replacement
`mean()/median()/mode()`

```
missing_values.py

df.isnull().sum()           # detect
df.dropna(inplace=True)
df['age'].fillna(df['age'].mean(),
inplace=True)
```

BEFORE → AFTER

StudentID	Name	Marks (before)	Marks (after)
I01	Aditi	78	78
I02	Rohan	-	76.4
I03	Meera	65	65
I04	Kabir	-	76.4
I05	Sana	88	88

Missing marks (-) are imputed using the column mean (76.4), preserving row count for downstream modeling.

Selection, Filtering & Sorting

SAMPLE DATASET - employees.csv

Name	Department	Salary	Age
Aditi	Data Science	62000	27
Rohan	Marketing	48000	31
Meera	Data Science	71000	29
Kabir	Finance	55000	35
Sana	Data Science	67000	26

1

Selection

Pick specific columns to work with.
`df[['Name', 'Salary']]`

2

Filtering

Keep only rows matching a condition.
`df[df['Salary'] > 60000]`

3

Sorting

Order rows by one or more columns.
`df.sort_values('Salary', ascending=False)`

select_filter_sort.py

```
# Selection: choose specific columns
```

```
subset = df[['Name', 'Salary']]
```

```
# Filtering: keep rows above a threshold
```

```
high_earners = df[df['Salary'] > 60000]
```

```
# Multi-condition filtering
```

```
ds_team = df[(df['Department'] == 'Data Science')  
             & (df['Age'] < 30)]
```

```
# Sorting: highest salary first
```

```
df.sort_values('Salary', ascending=False)
```

Grouping & Ranking

groupby() — Split · Apply · Combine

Groups rows sharing a common value, applies an aggregation (sum, mean, count) to each group, then combines results into a summary table.

Split



Apply



Combine

Ranking assigns a relative order within a group - useful for leaderboards, top-N selection, and percentile analysis.

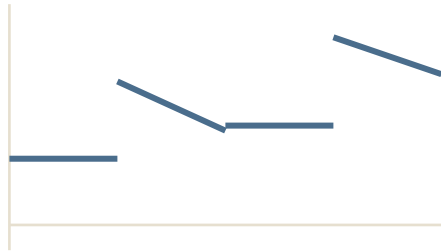
Region	Total Sales	Avg Order	Rank
South	₹ 8.4L	₹ 4,200	1
West	₹ 7.1L	₹ 3,950	2
North	₹ 6.6L	₹ 3,800	3
East	₹ 5.2L	₹ 3,400	4

```
df['Rank'] =  
df.groupby('Dept')['Sales'].rank(ascending =  
False)
```

```
groupby_rank.py  
  
summary = (df.groupby('Region')  
            .agg(Total_Sales=('Sales', 'sum'),  
                 Avg_Order=('Order', 'mean'))  
  
summary['Rank'] =  
summary['Total_Sales']\.  
rank(ascending=False).ast  
ype(int)
```

Data Visualization with Matplotlib

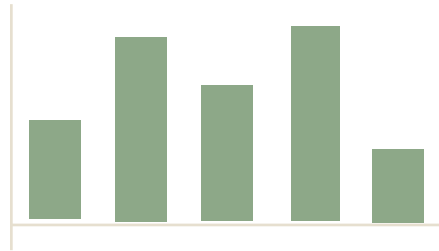
Numbers hide patterns that charts reveal instantly - trends, outliers, and distributions become visible only once data is plotted.



Line Chart

```
plt.plot(x, y)
```

Trends over time / sequence



Bar Chart

```
plt.bar(x, y)
```

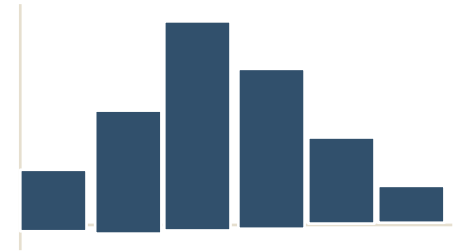
Comparing categories



Scatter Plot

```
plt.scatter(x, y)
```

Relationship between 2 variables



Histogram

```
plt.hist(data)
```

Distribution of one variable

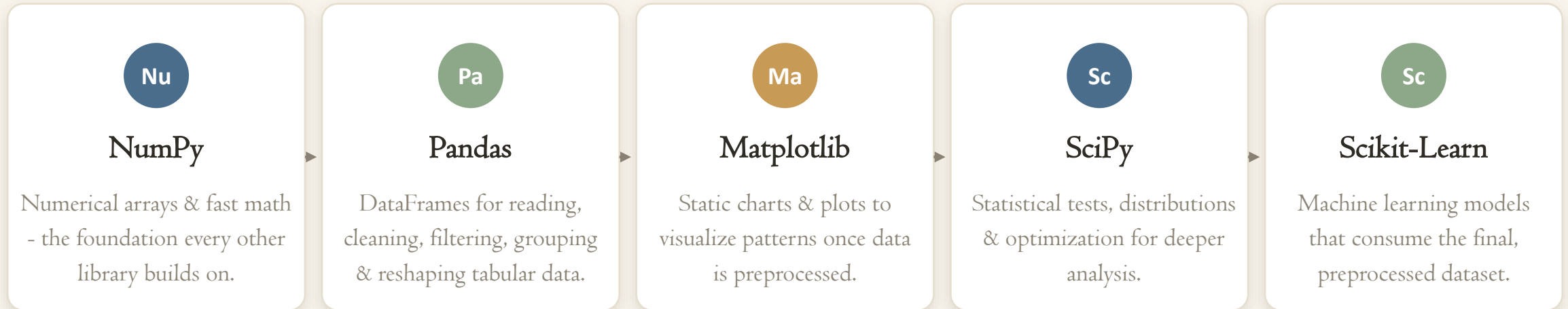
visualize.py

```
import matplotlib.pyplot as plt # after preprocessing is complete

df['Marks'].plot(kind='hist', bins=10, color='steelblue')
plt.title('Distribution of Student Marks'); plt.xlabel('Marks'); plt.show()
```


The Python Data Science Ecosystem

Each library plays a distinct role - together they form one connected pipeline, not five separate tools.



HOW THEY CONNECT

NumPy arrays power Pandas DataFrames → Pandas cleans & structures the data → Matplotlib visualizes it → SciPy validates statistical assumptions → Scikit-Learn trains models on the final, ML-ready dataset.

Preprocessing (Pandas + NumPy) is the largest and most critical stage in this chain — it determines everything downstream.

Mini Case Study: The Titanic Dataset



```
titanic_pipeline.py

df = pd.read_csv('titanic.csv')
df.drop(columns=['Cabin', 'Ticket'], inplace=True)
df['Age'].fillna(df['Age'].median(), inplace=True)
df = df[df['Fare'] > 0]
survival = df.groupby(['Pclass', 'Sex'])['Survived'].mean()
survival.plot(kind='bar', color='#4A6D8C')
```

KEY INSIGHT

First-class female passengers had a survival rate above 90%, versus under 20% for third-class male passengers — a pattern invisible in the raw table but obvious once grouped and plotted.

This single narrative touches cleaning, missing values, filtering, grouping and visualization — every concept from this talk.

Common Challenges & Best Practices

CHALLENGES

-  **Missing Values**
Incomplete records skew statistics and confuse models
-  **Duplicates**
Repeated rows inflate importance and bias aggregates
-  **Noise**
Random errors and typos obscure genuine patterns
-  **Outliers**
Extreme values distort means and model boundaries
-  **Inconsistent Formatting**
Mixed date formats, casing & units break joins

BEST PRACTICES

-  **Validate Data**
Check types, ranges & schema before every transformation
-  **Keep Backups**
Preserve the raw dataset untouched for reproducibility
-  **Automate Preprocessing**
Wrap steps in reusable functions or pipelines
-  **Document Transformations**
Record every change for auditability and trust

From Raw Data to Machine Learning-Ready



Data preprocessing turns messy, real-world data into a disciplined, trustworthy foundation - the single most decisive stage of any machine learning project.

THANK YOU!